OFFICIAL DEFENSE DOSSIER | VIVA EXAMINATION 2026

CyberShield & Codexa Security Platform

Complete Technical Architecture & Viva Defense Presentation Manual

CANDIDATE / OPERATOR	PRIMARY ASSIGNMENT	SECURITY GRADE
Jainam (Codewithjainam7)	**CyberShield Frontend & Codexa Full-Stack**	**GRADE A+ (HARDENED)**

The 45-Second "Hero" Presentation Speech

MEMORIZE FOR VIVA

*"Respected examiners, good morning. Our project establishes an end-to-end cybersecurity ecosystem across two vital fronts: **Live Perimeter Defense (CyberShield)** and **Pre-Deployment Static Security Auditing (Codexa)**.*

*In **CyberShield**, I led the frontend architecture: building a zero-dependency Canvas 2D 3D Threat Projection Globe, real-time heuristic scam detection, k-anonymity credential breach auditing, and automated PDF dossier generation.*

*In **Codexa**, I engineered the full-stack system: a **Java 21 / Spring Boot 3** deterministic AST (Abstract Syntax Tree) engine that analyzes 50,000 files with zero code execution risk, paired with a **React + Tailwind** interactive sandbox delivering line-level OWASP fixes and CycloneDX SBOM blast-radius graphs.*

Together, they create a complete closed-loop defense: hardening source code before production and protecting runtime assets after deployment."

1. CyberShield: Frontend Engineering Highlights

REACT 19 + VITE 8

- 3D Interactive Threat Projection Globe (Canvas 2D): Pure mathematical 3D projection without Three.js overhead. Converts spherical coordinates into 3D Cartesian coordinates via Euler yaw/pitch rotation matrices. Handles 1,200+ continental coordinates with depth shading, 3D missile trajectories, and high-DPI scaling.
- Compromised Identity Auditor (k-Anonymity): Secure credential leak lookup where passwords never leave the browser. Client computes SHA-1 and sends only first 5 hex characters; local matching verifies exposure against 15+ billion compromised records.
- Link & Scam Heuristic Auditor: Analyzes phishing homoglyphs (e.g. Cyrillic replacements), urgency psychological triggers, and illicit UPI banking verbs.
- Executive Client-Side PDF Generator: Client-side dynamic report synthesis via `jsPDF` and `html2canvas` for formal SOC technical audit exports.
- Cinematic Neural Bridge: Full-screen quantum warp transition seamlessly connecting CyberShield into Codexa Cloud.

`React 19` `Vite 8` `HTML5 Canvas 2D` `k-Anonymity SHA-1` `jsPDF` `Lucide Vectors`

2. Codexa: Full-Stack Architecture (Backend & Frontend) JAVA 21 + SPRING BOOT 3 + REACT 18

Core Purpose: Autonomous SAST & Production-Readiness Gatekeeper answering: *"Can this codebase safely deploy to production?"*

- Deterministic AST Analysis Engine (Java 21): Uses typed Abstract Syntax Trees (JavaParser + Polyglot rules) across 35+ rules. Evaluates real data flows rather than brittle regex pattern matching.
- Zero-Code-Execution Sandbox: Pure syntactic/lexical evaluation without classloading or bytecode execution, preventing malicious repo exploits.
- Industry Standards Output: Produces **SARIF v2.1.0** (compatible with GitHub Security tab) and **CycloneDX v1.5 / SPDX SBOM** with transitive CVE blast radius graphs.
- Interactive Security Playground (Frontend): In-browser code editor with real-time rule triggers and 1-Click AI Fix generating parameterized `PreparedStatement` diffs.
- Production Readiness Score (0–100): Weighted across Security (40%), Quality (25%), Dependencies (20%), and Hardening (15%).

`Java 21 LTS` `Spring Boot 3.3.3` `AST JavaParser` `SARIF v2.1.0` `CycloneDX v1.5` `React 18 + Tailwind`

Top 10 Expected Viva Questions & High-Scoring Answers CRITICAL FOR EVALUATION

Q1. Why Java 21 & Spring Boot for Codexa instead of Python?

Answer: High-throughput static analysis on large repos (up to 3 GB / 50,000 files) requires strict type safety and high parallelism. Java 21's **Virtual Threads (Project Loom)** enable parsing thousands of file ASTs concurrently with minimal memory, without the thread blocking of Python's Global Interpreter Lock (GIL).

Q2. What is an Abstract Syntax Tree (AST), and why not use Regex?

Answer: An AST is a typed hierarchical tree of code grammar. Regex linters produce high false positives because they are fooled by comments or variable reassignments. An AST understands program semantics—it verifies if an argument to `Statement.executeQuery()` was tainted by untrusted user input (e.g. `request.getParameter()`).

Q3. How does k-Anonymity protect user passwords during breach auditing?

Answer: The client calculates the SHA-1 hash locally and sends only the **first 5 hexadecimal characters** to the server. The API returns ~500 matching hash suffixes. The client performs the full 40-character match locally. The plaintext password never leaves the user's browser.

Q4. How does the 3D Vector Threat Globe work in Canvas 2D without Three.js?

Answer: Converts spherical lat/lon coordinates into 3D Cartesian coordinates via trigonometry: $x = r \cos(\phi)\sin(heta)$, $y = r \sin(\phi)$, $z = r \cos(\phi)\cos(heta)$. Then applies Euler rotational transformation matrices. Points with $z \ge 0$ render on the front hemisphere with depth shading; points with $z < 0$ render faintly on the back hemisphere.

Q5. How does Codexa ensure safety against malicious repositories?

Answer: Codexa performs strictly **lexical and syntactic parsing**. It never compiles or executes untrusted code, eliminating arbitrary remote code execution risks. It runs inside a non-root hardened Docker container with ephemeral sandbox directories.

Q6. What is SARIF v2.1.0, and why is it essential?

Answer: SARIF (Static Analysis Results Interchange Format) is an OASIS standard accepted by GitHub Code Scanning, SonarQube, and CI/CD tools. Codexa's SARIF output allows enterprise developers to see vulnerabilities directly inside GitHub pull request review tabs.

Q7. What is an SBOM and a CycloneDX graph?

Answer: An SBOM (Software Bill of Materials) is a machine-readable inventory of all direct and transitive third-party libraries. CycloneDX v1.5 is the OWASP standard format. Codexa graphs the dependency blast radius to reveal which services are exposed when a transitive dependency has a zero-day CVE.

Q8. How does CyberShield and Codexa create a complete defense loop?

Answer: **Defense-in-Depth.** Codexa secures the *Pre-Runtime / Pipeline* layer (stopping vulnerable code and leaked keys before merge). CyberShield secures the *Runtime / Perimeter* layer (monitoring live attack vectors, auditing leaked employee credentials, and analyzing phishing links).

Q9. How is the Production Readiness Score (0-100) calculated?

Answer: Weighted mathematical formula: **Security Vulnerabilities (40%)**, **Structural AST Code Quality (25%)**, **Dependency CVE Hygiene (20%)**, and **Operational Hardening (15%)**.

Q10. What was your biggest frontend engineering challenge?

Answer: Rendering the 3D Vector Threat Globe on raw Canvas 2D with full mobile responsiveness—managing high-DPI scaling via `devicePixelRatio`, smooth touch-drag rotation without triggering page scroll stutter, and maintaining 60fps missile trajectories.
